E Commerce Product and Order management System.

Abstract

The E-Commerce Product and Order Management System is a web-based application designed to manage products, customers, and orders efficiently. The system allows administrators to add, update, delete, and view products while customers can browse products, place orders, and track order status.

MongoDB is used as the database because it provides flexibility, scalability, and high performance for storing product and order information in JSON-like documents.

Objectives

- Manage product inventory.
- Store customer details.
- Create and manage orders.
- Track order status.
- Generate sales reports.
- Provide secure database storage using MongoDB.
- Implement REST APIs for communication.

Product Collection

```
{
  "_id": "ObjectId",
  "name": "Laptop",
  "description": "Dell Inspiron 15",
  "price": 55000,
  "stock": 10,
  "category": "Electronics",
  "image": "laptop.jpg",
  "createdAt": "2025-01-01"
}
```

User Collection

```
{
  "_id": "ObjectId",
  "name": "John Doe",
  "email": "john@gmail.com",
  "password": "hashedpassword",
  "role": "customer"
}
```

Order Collection

```
{
  "_id": "ObjectId",
  "customerId": "ObjectId",
  "products": [
    {
      "productId": "ObjectId",
      "quantity": 2
    }
  ],
  "totalAmount": 110000,
  "status": "Pending",
  "orderDate": "2025-01-01"
}
```

Folder Structure

```
ecommerce-project/
│
├── models/
│   ├── Product.js
│   ├── User.js
│   └── Order.js
│
├── routes/
│   ├── productRoutes.js
│   ├── userRoutes.js
│   └── orderRoutes.js
│
├── controllers/
│   ├── productController.js
│   └── orderController.js
│
├── config/
│   └── db.js
│
├── server.js
│
└── package.json
```

MongoDB Connection Code

```
const mongoose = require('mongoose');

const connectDB = async () => {
    try {
        await mongoose.connect(
            'mongodb://localhost:27017/ecommerceDB'
        );

        console.log('MongoDB Connected');
    } catch (error) {
        console.error(error);
        process.exit(1);
    }
};

module.exports = connectDB;
```

Product Model

```
const mongoose = require('mongoose');

const ProductSchema = new mongoose.Schema({
    name: {
        type: String,
        required: true
    },

    description: {
        type: String,
        required: true
    },

    price: {
        type: Number,
        required: true
    },

    stock: {
        type: Number,
        default: 0
    },

    category: {
        type: String
    },

    image: {
        type: String
    }

}, { timestamps: true });
```

Product Controller

```
const Product = require('../models/Product');

exports.addProduct = async (req, res) => {

    try {

        const product = await Product.create(req.body);

        res.status(201).json(product);

    } catch (error) {

        res.status(500).json({
            message: error.message
        });
    }
};

exports.getProducts = async (req, res) => {

    const products = await Product.find();

    res.json(products);
};

exports.updateProduct = async (req, res) => {

    const product = await Product.findByIdAndUpdate(
        req.params.id,
        req.body,
        { new: true }
    );

    exports.deleteProduct = async (req, res) => {

        await Product.findByIdAndDelete(
            req.params.id
        );

        res.json({
            message: 'Product Deleted'
        });
    };
```

Future Enhancements

Online Payment Gateway Integration.
Email Notifications.
Product Reviews and Ratings.
Shopping Cart Module.
Discount Coupon System.
Admin Dashboard Analytics.
Inventory Forecasting.
AI-Based Product Recommendations

Conclusion

The **E-Commerce Product and Order Management System** is a complete web application developed using **MongoDB, Node.js, Express.js, and React.js**. It efficiently manages products, customers, inventory, and orders while providing scalable and flexible data storage through MongoDB. This project demonstrates CRUD operations, RESTful API development, database management, and modern web application architecture, making it suitable for academic projects, final-year projects, and practical learning of MERN stack development.